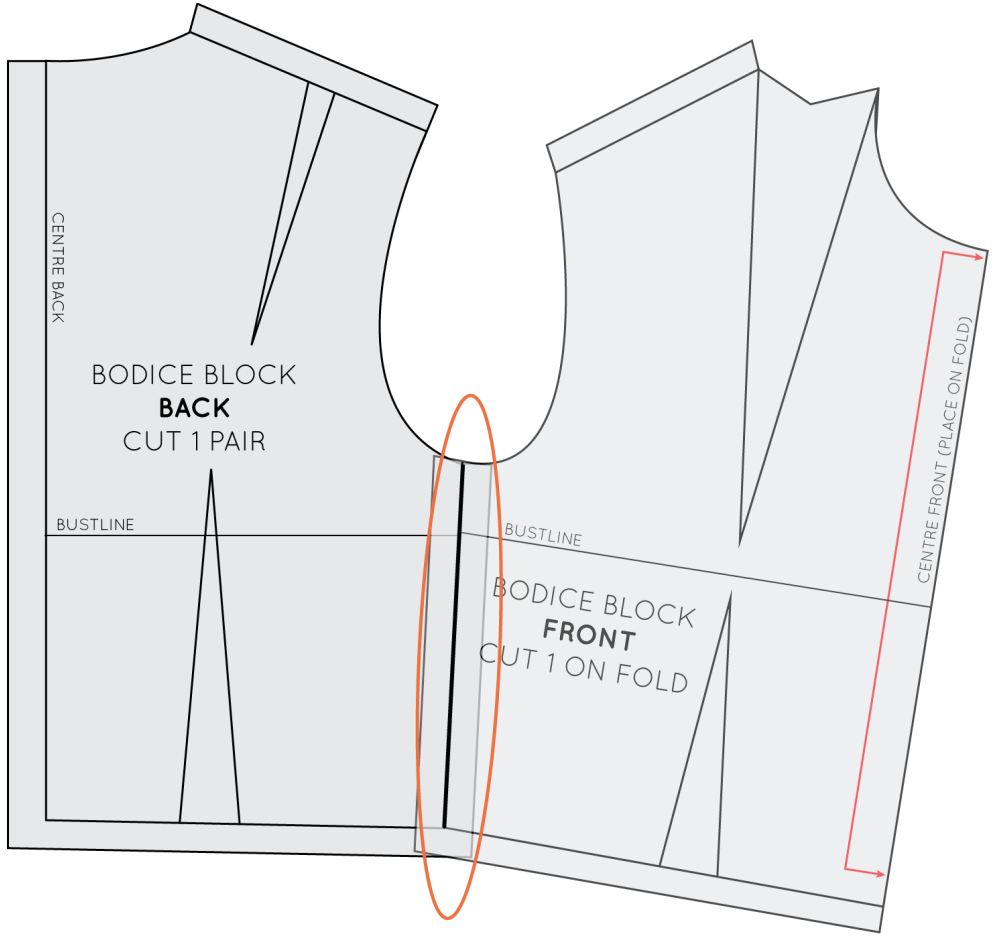
University of Exeter

ECM2433: The C Family
C++ templates and STL

Overview

- Templates
  - macro
  - function templates
  - class templates
- Standard template library
  - containers
  - iterators



Swap example

Some overloaded functions are very generic, i.e. they could be applied to almost any datatype:

We would have to create separate functions for each datatype

But the code in each separate function would otherwise be identical

Laborious and hard to maintain

What if we could keep the logic, but change the type?

```
void myswap (int &x, int &y)
{
    int temp = x;
    x = y;
    y = temp;
}
void myswap (float &x, float &y)
{
    float temp = x;
    x = y;
    y = temp;
}
void myswap (double &x, double &y)
{
    double temp = x;
    x = y;
    y = temp;
}
```

Swap example: before templates

University of Exeter

```
#include<iostream>
#include "swap.h"

MYSWAP(int);
MYSWAP(float);

void myswap (int &x, int &y)
{
        int temp = x;
        x = y;
        y = temp;
} ;

void myswap (float &x, float &y)
{
        float temp = x;
        x = y;
        y = temp;
} ;
```

```
#define MYSWAP(MY_TYPE) \
void myswap (MY_TYPE &x, MY_TYPE &y) \
{ \
        MY_TYPE temp = x; \
        x = y; \
        y = temp; \
}
```

swap.h

Preprocessor Macro
Runs before compiler
Text substitution

Swap example: before templates

```
#include<iostream>
#include "swap.h"

MYSWAP(int);
MYSWAP(float);
MYSWAP(double);

int main()
{

    int a = 10, b = 20;
    myswap(a, b);
    std::cout << "a: " << a << ", b: " << b << std::endl;

    float f1 = 1.5f, f2 = 2.5f;
    myswap(f1, f2);
    std::cout << "f1: " << f1 << ", f2: " << f2 << std::endl;

    double x = 1.5, y = 2.5;
    myswap(x, y);
    std::cout << "x: " << x << ", y: " << y << std::endl;


    return 0;
}
```

```
#define MYSWAP(MY_TYPE) \
void myswap (MY_TYPE &x, MY_TYPE &y) \
{ \
        MY_TYPE temp = x; \
        x = y; \
        y = temp; \
}
```

swap.h

```
a: 20, b: 10
f1: 2.5, f2: 1.5
x: 2.5, y: 1.5
```

Problems with macros

```
#include "swap.h"

MYSWAP(int);
```

main.cpp

```
#include "swap.h"

MYSWAP(int);
```

utils.cpp

```
[Linker Error]: multiple definition of
'void myswap(int&, int&)'
first defined here: main.o
second defined here: utils.o
```

Fail to support modular compilation.

Problems with macros

University of Exeter

Another swap with macro

```
#include <iostream>

#define SWAP(a,b) \
{auto temp = a; a = b; b = temp;}

int main()
{

    int a = 10;
    float b = 1.5f;
    SWAP(a,b)
    std::cout << "a: " << a << ", b: "
    << b << std::endl;

    return 0;
}
```

```
int main()
{

    int a = 10;
    float b = 1.5f;
    auto temp = a; a = b; b = temp;
    std::cout << "a: " << a << ", b:
    " << b << std::endl;

    return 0;
}
```

```
a: 1, b: 10
```

Require careful design to ensure type safety

Function templates

How to build a family of similar looking functions

This is a template. It's not a type

T gets replaced with a specific type

```
template<typename T>
void myswap (T &x, T &y)
{
    T temp = x;
    x = y;
    y = temp;
}
```

T gets replaced when myswap is instantiated

```
void myswap (int &x, int &y)
{
    int temp = x;
    x = y;
    y = temp;
}
void myswap (float &x, float &y)
{
    float temp = x;
    x = y;
    y = temp;
}
void myswap (string &x, string &y)
{
    string temp = x;
    x = y;
    y = temp;
}
```

Function templates

University of Exeter

```
template<typename T>
void myswap (T &x, T &y)
{
    T temp = x;
    x = y;
    y = temp;
}
```

```
int main ()
{
    int   a = 1;
    int   b = 2;
    cout << "a=" << a << ", b=" << b << endl;
    myswap(a,b);
    cout << "a=" << a << ", b=" << b << endl;

    string   c = "elephants";
    string   d = "penguin";
    cout << "c=" << c << ", d=" << d << endl;
    myswap(c,d);
    cout << "c=" << c << ", d=" << d << endl;

    return 0;
}
```

```
a = 1, b = 2
a = 2, b = 1
c = elephants, d = penguin
c = penguin, d = elephants
```

Function templates

```
template<typename T>
void myswap (T &x, T &y)
{
    T temp = x;
    x = y;
    y = temp;
}
```

swap.h

```
#include <iostream>
#include "swap.h"

int main()
{


    int a = 10;
    float b = 1.5f;
    myswap(a,b)
    std::cout << "a: " << a << ", b: "
    << b << std::endl;


    return 0;
}
```

error: no matching function for call to 'myswap'

Type checked at compile time

Stack example

Our class called Stack is not really “Stack”,
it is “IntegerStack”

```
int main ()
{
    Stack s;
    s.push(23);
    s.push(42);
}
```

```
// Define the Stack class
class Stack
{
        int     stck[SIZE];
        int     index;
    public:
        void    push(int);
        int     pop(void);
};
```

Stack example

University of Exeter

What if we want to use a stack datastructure for lots of different datatypes?

```
int main ()
{
   // declare variables
   IntegerStack is;
   StringStack  ss;
   MyClassStack mcs;
   MyClass      mc;

   // push data onto stacks
   is.push(23);
   ss.push("elephants");
   mcs.push(mc);
}
```

We would have to create separate classes for each datatype

But the code in each separate class would otherwise be identical

Laborious and hard to maintain

Class templates

How to build a family of similar looking classes

```
template<typename T>
class Stack
{
        T     stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(T);
        T       pop();
};
```

This is a template. It's not a type

```
class Stack
{
        int   stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(int);
        int     pop();
};
```

Class templates

How to build a family of similar looking classes

```
template<typename T>
class Stack
{
        T     stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(T);
        T       pop();
};
```

```
Stack<int> s1;
s1.push(23);
cout << s1.pop() << endl;

Stack<string> *s2 = new Stack<string>;
s2->push("Hello");
cout << s2->pop() << endl;
delete s2;
```

Class templates

University of Exeter

```
template<typename T>
class Stack
{
        T     stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(T);
        T       pop();
};
```

```
Stack<int> s1;
s1.push(23);
cout << s1.pop() << endl;
```

Compiler produces code like this …

```
class Stack<int>
{
        int   stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(int);
        int     pop();
};
```

Class templates

```
template<typename T>
class Stack
{
        T     stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(T);
        T       pop();
};
```

```
Stack<string> *s2 =
new Stack<string>;
```

Similarly, compiler produces …

```
class Stack<string>
{
        string stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(string);
        string  pop();
};
```

Code for a specific type is generated on-demand, when you use it

Template class implementation

When you implementing a template, you may try

```
template<typename T>
class Stack
{
        T     stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(T);
        T       pop();
};
```

stack.h

```
T Stack::pop(){
    // implementation …

}
```

Compiler does not know what T is!

stack.h

Template class implementation

General practice is to place both the interface and implementation of templates in a header file.

```
template<typename T>
class Stack
{
        T     stck[SIZE];
        int   index;
    public:
        static int count;
        Stack();
        Stack(Stack &);
        ~Stack();
        void    push(T);
        T       pop();
};
```

stack.h

```
template<typename T>
T Stack<T>::pop(){
        // implementation …



}
```

stack.h

When use with template `Stack` is not a type, but `Stack<T>` is

Standard templates library

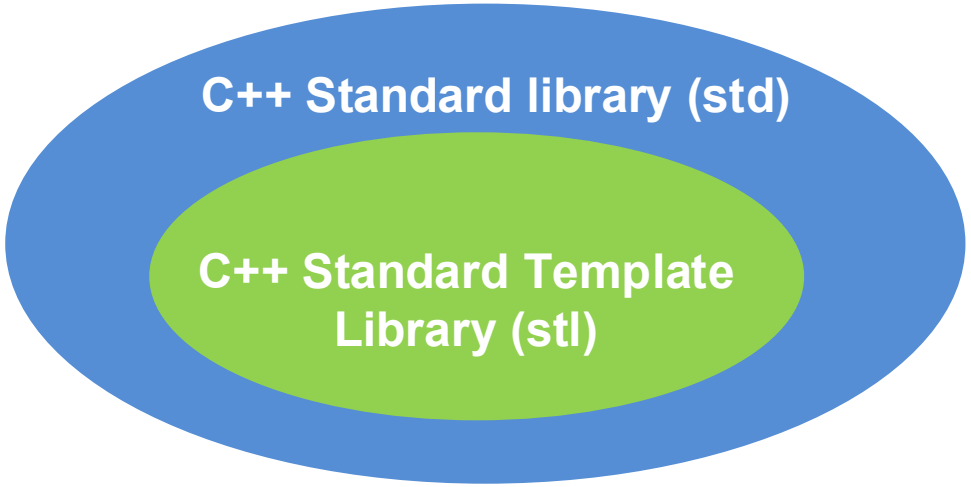
Some data structures, such

as stacks

queues

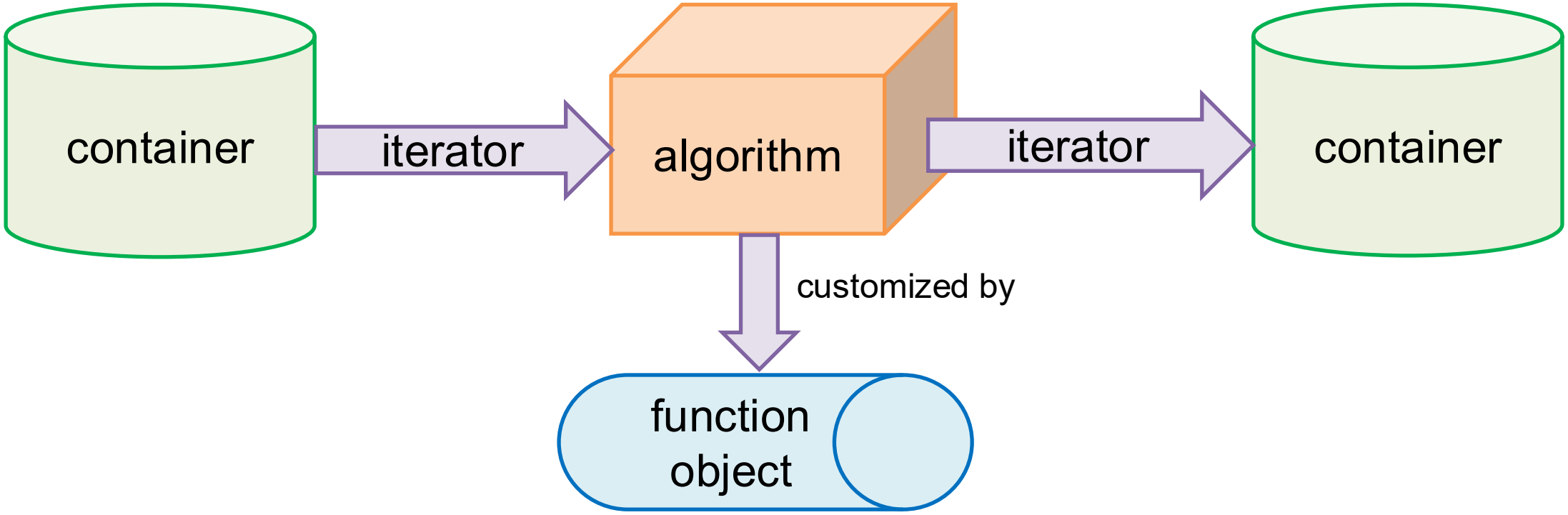
are so widely used that C++ provides a set of standard templates through the

Standard Template Library



STL: standard template library

Four main components:




```
std::sort(v.begin(), v.end(), &func });
```

STL: containers

Containers store a collection of elements

Sequence containers: store a linear sequence of elements

- array, vector, deque, forward_list, list

Associative containers: organize elements by unique items

- Ordered
  - set, multiset, map, multimap
- Unordered
  - unordered_set, unordered_multiset, unordered_map, unordered_multimap

Container adapters:

- stack, queue, priority_queue

STL: container: stack

After all your C and C++ stack coding, the STL provides a stack template for you!

https://www.cplusplus.com/reference/stack/stack/

Member functions

(constructor)	Construct stack (public member function)
empty	Test whether container is empty (public member function)
size	Return size (public member function)
top	Access next element (public member function)
push	Insert element (public member function)
emplace C++11	Construct and insert element (public member function)
pop	Remove top element (public member function)
swap C++11	Swap contents (public member function)

on the website, click on each of these to get example code

STL: container: stack

University of Exeter

```
#include <stack>
std::stack<int> si;
si.push(43);
si.push(99);
std::cout << "Size: " << si.size() << std::endl;
std::cout << "Top : " << si.top()  << std::endl;
si.pop();
std::cout << "Top : " << si.top()  << std::endl;
std::cout << "Size: " << si.size() << std::endl;
```

```
Size: 2
Top : 99
Top : 43
Size: 1
```

```
std::stack<float> sf;
sf.push(5.86);

std::stack<MyClass> st;
MyClass m;
st.push(m);
```

fx Member functions

(constructor)	Construct stack (public member function)
empty	Test whether container is empty (public member function)
size	Return size (public member function)
top	Access next element (public member function)
push	Insert element (public member function)
emplace C++11	Construct and insert element (public member function)
pop	Remove top element (public member function)
swap C++11	Swap contents (public member function)

STL: container: vector

https://www.cplusplus.com/reference/vector/vector/

Vector

Vectors are sequence containers representing arrays that can change in size.

Just like arrays, vectors use contiguous storage locations for their elements, which means that their elements can also be accessed using offsets on regular pointers to its elements, and just as efficiently as in arrays. But unlike arrays, their size can change dynamically, with their storage being handled automatically by the container.

Internally, vectors use a dynamically allocated array to store their elements. This array may need to be reallocated in order to grow in size when new elements are inserted, which implies allocating a new array and moving all elements to it. This is a relatively expensive task in terms of processing time, and thus, vectors do not reallocate each time an element is added to the container.

STL: container: vector

https://www.cplusplus.com/reference/vector/vector/

Vector

Vectors are sequence containers representing arrays that can change in size.

Just like arrays, vectors use contiguous storage locations for their elements, which means that their elements can also be accessed using offsets on regular pointers to its elements, and just as efficiently as in arrays. But unlike arrays, their size can change dynamically, with their storage being handled automatically by the container.

Internally, vectors use a dynamically allocated array to store their elements. This array may need to be reallocated in order to grow in size when new elements are inserted, which implies allocating a new array and moving all elements to it. This is a relatively expensive task in terms of processing time, and thus, vectors do not reallocate each time an element is added to the container.

Note this, which you should consider if you choose to use this container

STL: container: vector

```
#include <vector>

std::vector<int> v;
v.push_back(43);
v.push_back(99);
v.push_back(0);

std::cout << "Vector v contains " << v.size() << " values." << std::endl;
std::cout << "The 2nd value is " << v[1] << std::endl;

std::cout << "The 3rd value is " << v[2] << std::endl;
v[2] = 5;
std::cout << "The 3rd value is " << v[2] << std::endl;
```

notice: operator[] overloading

```
Vector v contains 3 values.
The 2nd value is 99
The 3rd value is 0
The 3rd value is 5
```

STL: iterator

University of Exeter

```
std::vector<int> myvector;
std::vector<int>::iterator it;

for (int i = 1; i <= 5; i++)
   myvector.push_back(i);

std::cout << "myvector contains:";
for (it = myvector.begin(); it != myvector.end(); it++)
   std::cout << ' ' << *it;
std::cout << std::endl;
```

```
myvector contains: 1 2 3 4 5
```

STL: iterator

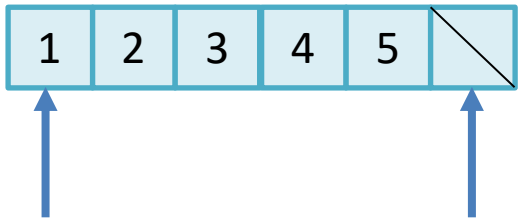
University of Exeter

```
std::vector<int> myvector;

for (int i = 1; i <= 5; i++)
   myvector.push_back(i);

std::cout << "myvector contains:";
for (auto it = myvector.begin(); it != myvector.end(); it++)
   std::cout << ' ' << *it;
std::cout << std::endl;
```

```
myvector contains: 1 2 3 4 5
```

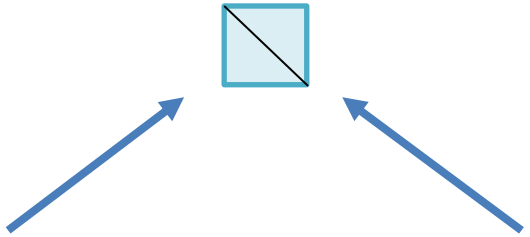
STL: iterator

```
std::vector<int> myvector;
std::vector<int>::iterator it;

for (int i = 1; i <= 5; i++)
    myvector.push_back(i);
```



`myvector.begin()` `myvector.end()`

Empty container



`myvector.begin() == myvector.end()`

STL: standard template library

University of Exeter

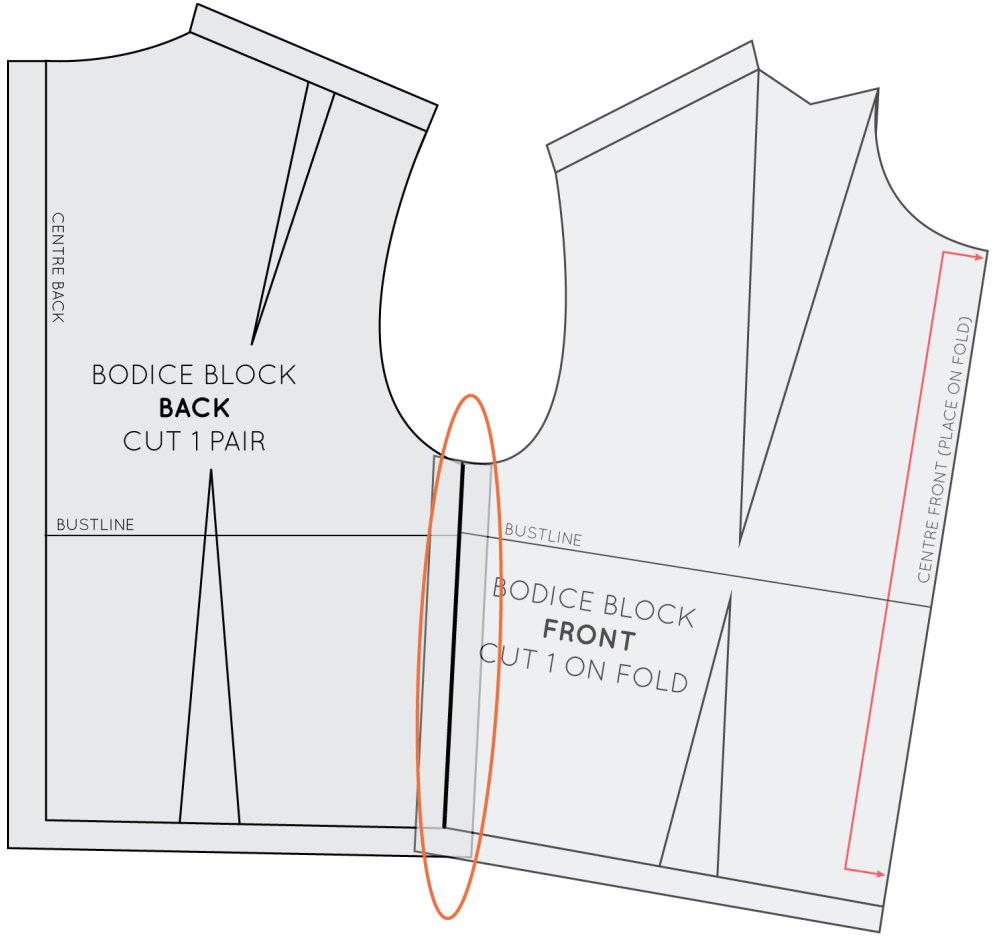
Four main components:

- containers
  - e.g. vector, queue, stack, list, map, unordered_map
  - https://en.cppreference.com/w/cpp/container
- iterators
  - objects that enable a program to traverse a container
  - https://en.cppreference.com/w/cpp/iterator
- algorithms
  - e.g. binary_search, sorting, counting
  - https://en.cppreference.com/w/cpp/algorithm
- function objects
  - https://en.cppreference.com/w/cpp/utility#General-purpose_utilities
  - https://en.cppreference.com/w/cpp/utility/functional

Key reference websites:

https://isocpp.org/get-started
links to:

https://en.cppreference.com/w/
site with its main focus on a complete Standard Template Library reference

Summary

- Templates
  - Function templates
  - Class templates
- Standard template library
  - Containers
  - Iterators